

```

import random

# Genetic Algorithm for OS Job Scheduling

# Input
n = int(input("Enter number of jobs: "))

jobs = []

for i in range(n):
    time = int(input(f"Enter execution time for Job {i + 1}: "))
    jobs.append(time)

# Parameters
population_size = 10
generations = 50
mutation_rate = 0.1

# Create a random schedule
def create_schedule():
    schedule = list(range(n))
    random.shuffle(schedule)
    return schedule

# Calculate fitness
def calculate_fitness(schedule):
    current_time = 0
    waiting_time = 0

    for job in schedule:
        waiting_time += current_time
        current_time += jobs[job]

    total_time = current_time

    # Higher fitness is better
    fitness = 1 / (1 + waiting_time)

```

```

        return fitness, waiting_time, total_time

# Create initial population
population = []

for i in range(population_size):
    population.append(create_schedule())

# Genetic Algorithm
for generation in range(generations):

    # Calculate fitness
    population.sort(
        key=lambda x: calculate_fitness(x)[0],
        reverse=True
    )

    # Select best half
    parents = population[:population_size // 2]

    new_population = parents.copy()

    # Crossover
    while len(new_population) < population_size:

        parent1 = random.choice(parents)
        parent2 = random.choice(parents)

        point = random.randint(1, n - 1)

        child = parent1[:point]

        for job in parent2:
            if job not in child:
                child.append(job)

        new_population.append(child)

```

```

# Mutation
for i in range(1, len(new_population)):

    if random.random() < mutation_rate:

        a = random.randint(0, n - 1)
        b = random.randint(0, n - 1)

        new_population[i][a], new_population[i][b] = \
            new_population[i][b], new_population[i][a]

population = new_population

# Get best schedule
population.sort(
    key=lambda x: calculate_fitness(x)[0],
    reverse=True
)

best_schedule = population[0]

fitness, waiting_time, total_time = calculate_fitness(best_schedule)

# CPU Efficiency
total_job_time = sum(jobs)

efficiency = (total_job_time / total_time) * 100

# Output
print("\n===== GENETIC ALGORITHM =====")

print("\nBest Job Schedule:")

for job in best_schedule:
    print(f"Job {job + 1} -> {jobs[job]} units")

```

```
print("\nMinimum Waiting Time:", waiting_time)
print("Total Execution Time:", total_time)
print("CPU Efficiency: {:.2f}%".format(efficiency))
print("Fitness: {:.6f}".format(fitness))

print("=====")
```

OUTPUT:

```
Enter number of jobs: 5
Enter execution time for Job 1: 10
Enter execution time for Job 2: 5
Enter execution time for Job 3: 8
Enter execution time for Job 4: 3
Enter execution time for Job 5: 6

===== GENETIC ALGORITHM =====

Best Job Schedule:
Job 4 -> 3 units
Job 2 -> 5 units
Job 5 -> 6 units
Job 3 -> 8 units
Job 1 -> 10 units

Minimum Waiting Time: 47
Total Execution Time: 32
CPU Efficiency: 100.00%
Fitness: 0.020833
```